Rapport de TP
Analyse d'une application Android avec code natif

UnCrackable Level 2

Rédigé pour : **Amira Ezbiri**

Filière : Cybersécurité

Sujet : Analyse statique d'un APK qui déplace la logique sensible dans une bibliothèque native via JNI

26 avril 2026

Table des matières

1 Contexte du laboratoire **2**

2 Environnement et outils **2**

3 Observation initiale de l'application **2**

4 Analyse Java avec JADX **3**

4.1 Classe `CodeCheck` 4

5 Extraction de la bibliothèque native **4**

6 Analyse du code natif avec Ghidra **5**

7 Validation finale **6**

8 Décodage lorsque la chaîne est masquée **7**

9 Résumé du flux de l'application **8**

10 Conclusion **8**

1 Contexte du laboratoire

Ce laboratoire porte sur une application Android volontairement piégée. L'interface semble simple : un champ de saisie et un bouton *VERIFY*. Pourtant, la vérification réelle n'est pas entièrement visible dans le code Java. La logique sensible est transférée vers une bibliothèque native chargée au runtime via JNI.

L'objectif pédagogique est de suivre le chemin de la donnée depuis l'interface jusqu'au code natif, puis de retrouver la chaîne secrète attendue par l'application.

2 Environnement et outils

L'analyse repose sur les outils suivants :

— **ADB** pour l'installation de l'APK sur l'émulateur ;

— **JADX** pour la décompilation du code Java ;

— **Ghidra** pour l'analyse statique de la bibliothèque `.so` ;

— **unzip** pour extraire le contenu de l'APK ;

— **Python 3** pour décoder puis inverser la chaîne finale.

3 Observation initiale de l'application

La première étape consiste à lancer l'application et à observer son comportement réel avant toute lecture du code.

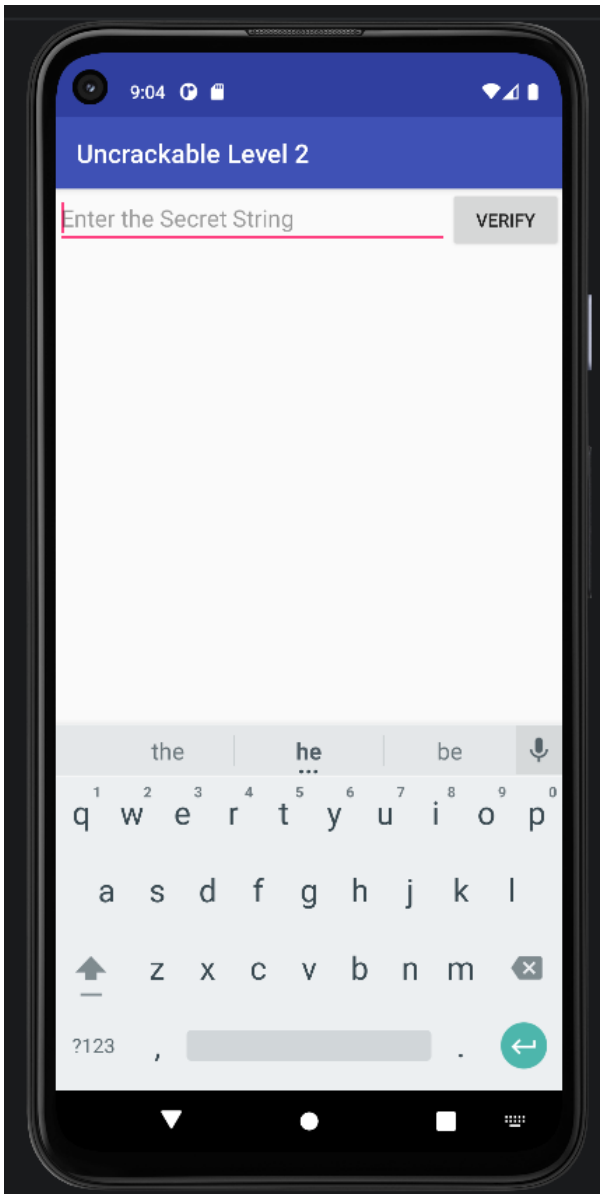

FIGURE 1 – Interface de départ de *UnCrackable Level 2*.

L'application présente une interface minimaliste. Cette observation est importante, car elle montre que le vrai sujet du TP n'est pas l'ergonomie, mais la logique de validation cachée derrière le bouton.

4 Analyse Java avec JADX

La décompilation de l'APK avec JADX permet de localiser rapidement `MainActivity`. C'est le point de départ de la logique côté interface.

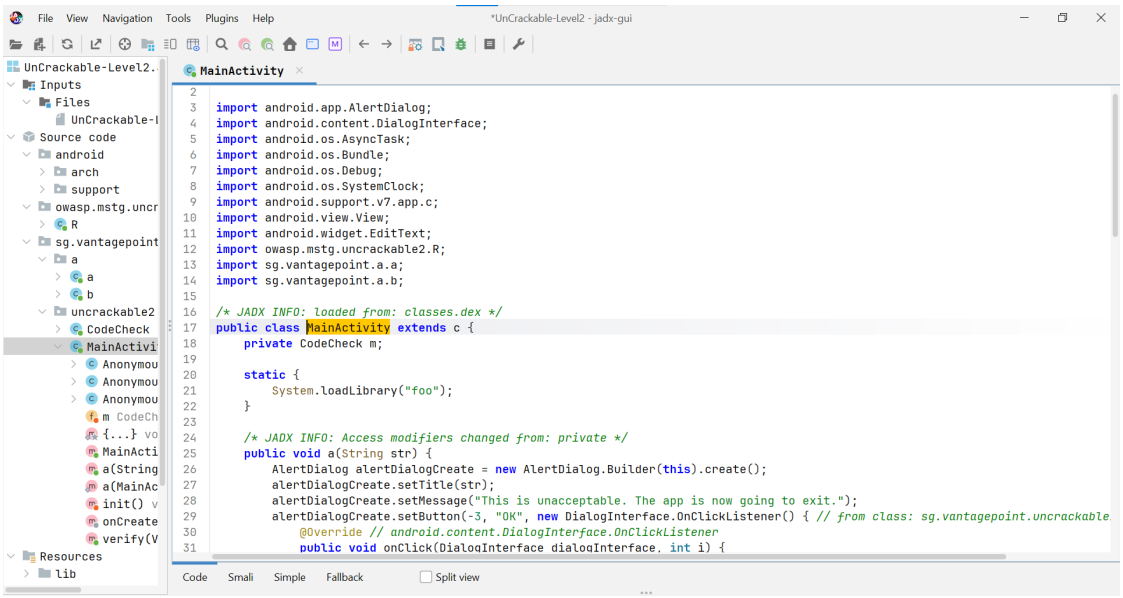

FIGURE 2 – Décompilation de `MainActivity` dans JADX.

Dans cette classe, on remarque l'appel à :

```
System.loadLibrary("foo");
```

Cet appel indique que l'application charge une bibliothèque native nommée `libfoo.so`.

4.1 Classe `CodeCheck`

La vérification n'est pas effectuée directement dans `MainActivity`. L'entrée utilisateur est transmise à une autre classe, `CodeCheck`, qui fait le lien avec le code natif.

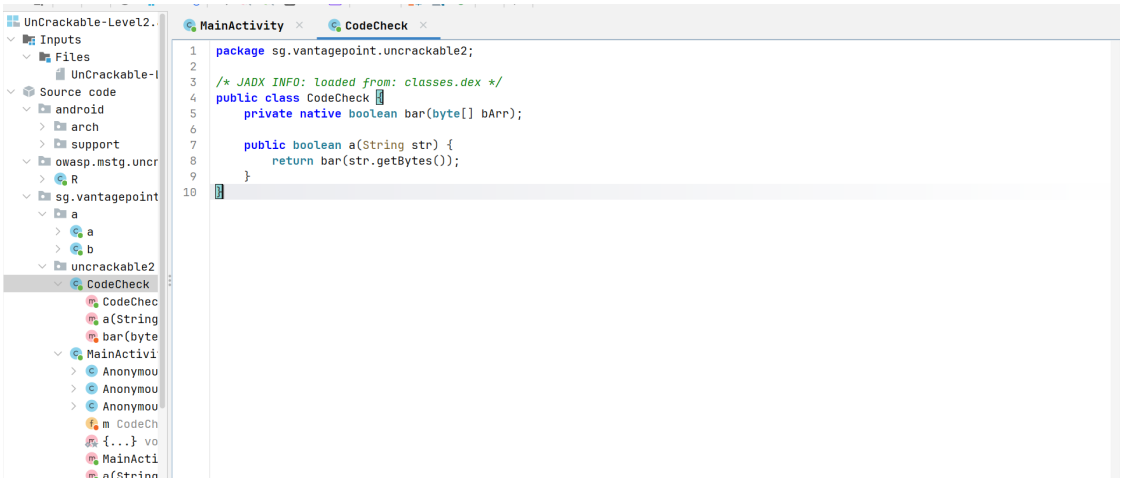

FIGURE 3 – Classe `CodeCheck` dans JADX avec la méthode native `bar`.

Le point essentiel ici est la présence du mot-clé `native`. Cela signifie que l'implémentation réelle de la méthode n'est pas en Java, mais dans une bibliothèque compilée en C/C++.

5 Extraction de la bibliothèque native

Un APK étant une archive, il suffit de le décompresser pour retrouver les fichiers embarqués. Le dossier `lib` contient plusieurs variantes selon l'architecture.

Le fichier à analyser est `libfoo.so`. Une capture du dossier le montre explicitement.

6 Analyse du code natif avec Ghidra

Une fois la bibliothèque ouverte dans Ghidra, l'analyse automatique permet de retrouver les symboles exportés et le pseudo-code décompilé.

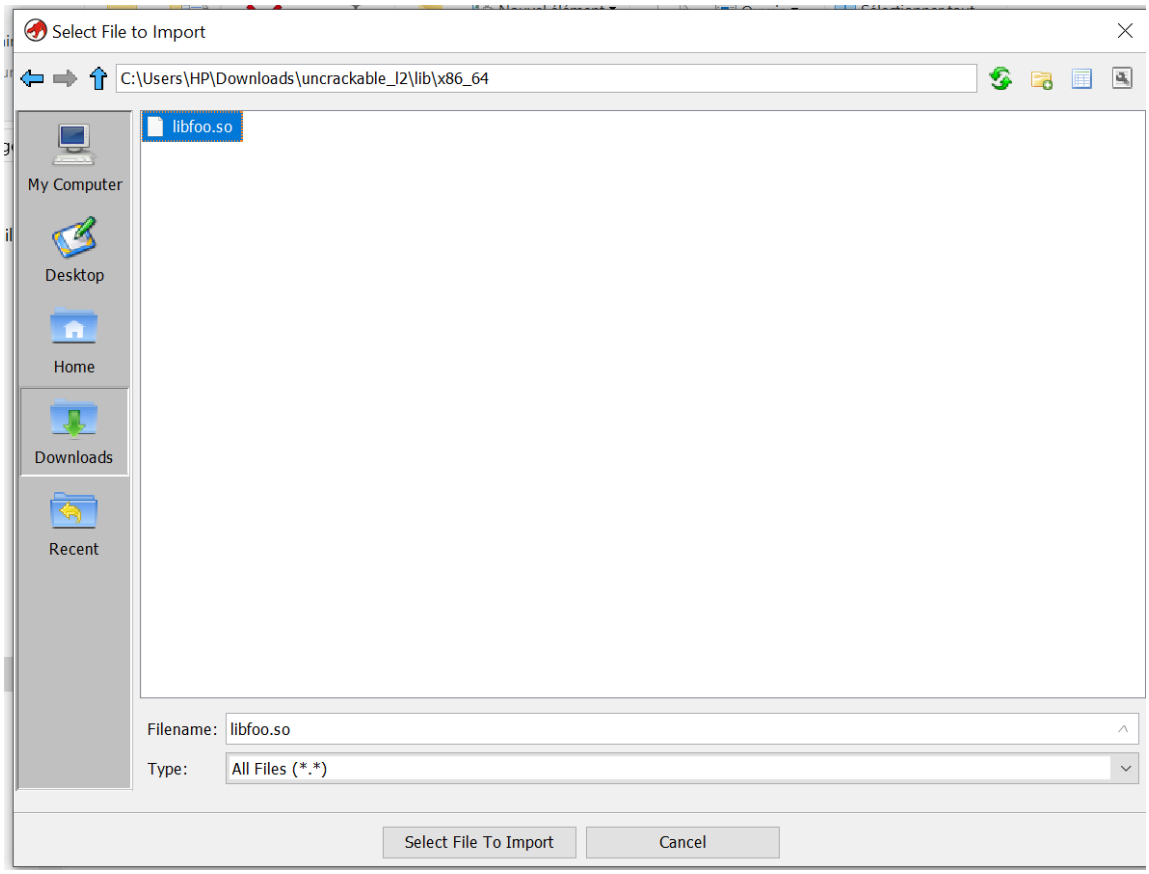

FIGURE 4 – Import de `libfoo.so` dans Ghidra.

La fonction JNI liée à `CodeCheck.bar` apparaît clairement avec un nom de type :

```
Java_sg_vantagepoint_uncrackable2_CodeCheck_bar
```

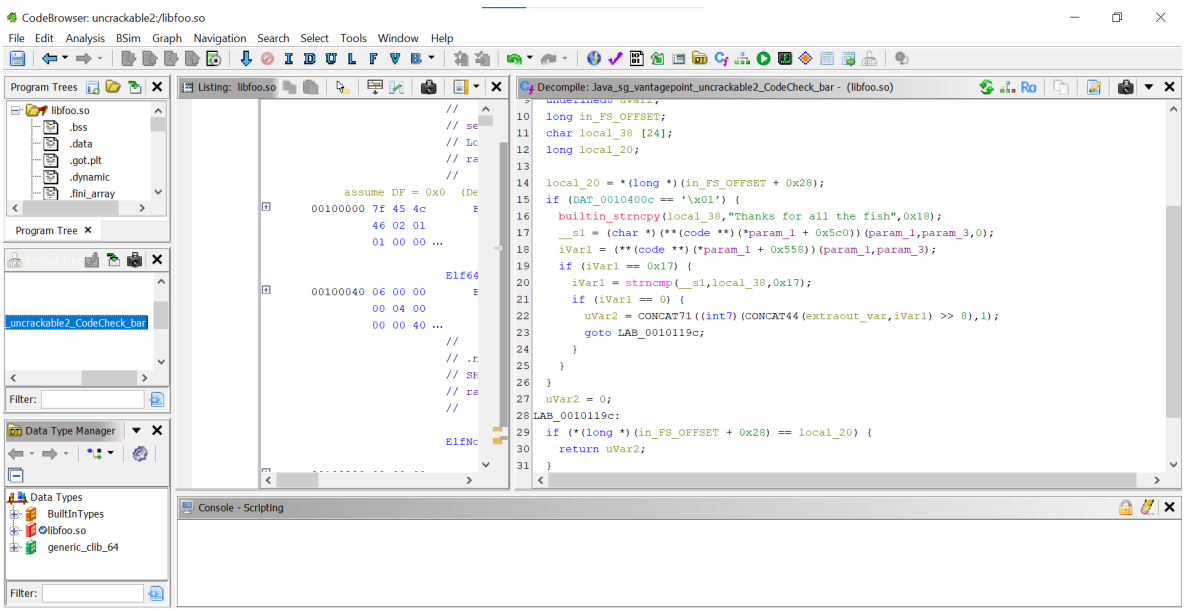

FIGURE 5 – Fonction JNI retrouvée dans Ghidra et pseudo-code décompilé.

Le pseudo-code montre une comparaison de chaînes avec `strncmp`. La chaîne de référence est directement visible dans le binaire. Dans ce challenge, elle correspond à :

```
Thanks for all the fish
```

7 Validation finale

Une fois la chaîne identifiée, il suffit de la tester dans l'application.

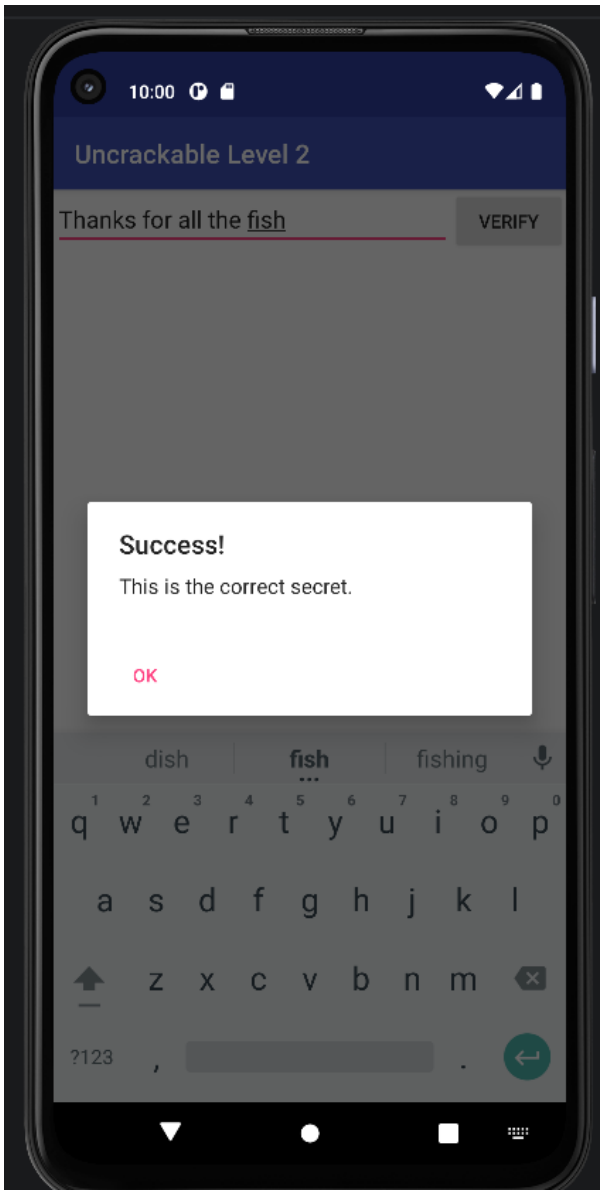

FIGURE 6 – Validation réussie avec la chaîne correcte.

Cette étape confirme que l'analyse statique a permis de retrouver le secret exact.

8 Décodage lorsque la chaîne est masquée

Dans certains cas, la chaîne n'est pas stockée en clair. Le TP montre alors une valeur encodée en hexadécimal ASCII. La logique de décodage est simple :

```
hex_data = "6873696620656874206c6c6120726f6620736b6e616854"
print(bytes.fromhex(hex_data).decode("ascii"))
```

On obtient :

```
hsif eht lla rof sknahT
```

Puis il suffit d'inverser la chaîne :

```
s = "hsif␣eht␣lla␣rof␣sknahT"
print(s[::-1])
```

Résultat final :

```
Thanks for all the fish
```

9 Résumé du flux de l'application

Utilisateur	→	saisit une chaîne dans l'interface
`MainActivity`	→	transmet la donnée à `CodeCheck`
`CodeCheck.a(...)`	→	appelle la méthode native `bar(...)`
`libfoo.so`	→	exécute la comparaison `strncmp`
Comparaison	→	succès si la chaîne est `Thanks for all the fish`

10 Conclusion

Ce laboratoire montre qu'une logique sensible déplacée dans une bibliothèque native n'est pas invisible pour autant. En suivant la donnée depuis l'interface jusqu'au binaire `.so`, puis en analysant la fonction JNI dans Ghidra, on retrouve le secret attendu sans difficulté majeure.

Pour Amira Ezbiri, ce TP constitue un très bon exercice d'initiation au reverse engineering Android : observer, suivre, extraire, analyser, puis valider.